# Middleware

The backend uses custom middleware for authentication, authorization, and security.

## Authentication Middleware

**File:** `backend/src/middleware/auth.js`

### `requireStaffOrAdmin`

Protects routes that require `staff` or `admin` roles.

```
const requireStaffOrAdmin = (req, res, next) => {
  const user = req.user;
  if (!user || !['staff', 'admin'].includes(user.role)) {
    return res.status(403).json({ success: false, message: 'Forbidden' });
  }
  next();
};
```

### `requireAdmin`

Protects routes that require the `admin` role.

```
const requireAdmin = (req, res, next) => {
  const user = req.user;
  if (!user || user.role !== 'admin') {
    return res.status(403).json({ success: false, message: 'Forbidden' });
  }
  next();
};
```

### Token Verification

JWT tokens are verified in the auth middleware:

1. Extract `Authorization` header
2. Verify JWT signature using `JWT_SECRET`
3. Attach decoded user payload to `req.user`
4. Handle expired or invalid tokens with 401 responses

## Security Middleware

**File:** `backend/src/middleware/security.js`

### `requestId`

Generates a unique request ID for distributed tracing:

```
const requestId = (req, res, next) => {
  req.id = req.headers['x-request-id'] || crypto.randomUUID();
  res.setHeader('x-request-id', req.id);
  next();
```

```
};
```

## `securityHeaders`

Sets security-related HTTP headers:

```
const securityHeaders = (req, res, next) => {
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('X-Frame-Options', 'DENY');
  res.setHeader('Referrer-Policy', 'strict-origin-when-cross-origin');
  res.setHeader('Permissions-Policy', 'camera=(), microphone=(), geolocation=()');
  res.setHeader('Cross-Origin-Resource-Policy', 'same-origin');
  res.setHeader('Cache-Control', 'no-store');
  next();
};
```

## `rateLimit`

Configures request rate limiting:

```
const rateLimit = (options) => {
  // options: { windowMs, max, keyPrefix }
};
```

## `validateProductionConfig`

Validates environment variables when `NODE_ENV=production`:

```
const validateProductionConfig = () => {
```

```
  // Enforces HTTPS CORS origins
  // Validates JWT_SECRET length
  // Ensures PUBLIC_TICKET_API_KEY is set
  // Validates TLS settings
};
```

# File Upload Middleware

Multer handles file uploads for ticket attachments:

```
const upload = multer({
  dest: process.env.UPLOAD_DIR || './uploads',
  limits: { fileSize: process.env.MAX_FILE_SIZE || 10485760 },
  fileFilter: (req, file, cb) => {
    const allowedTypes = (process.env.ALLOWED_FILE_TYPES ||
'jpg,jpeg,png,gif,pdf,doc,docx,xls,xlsx,zip').split(',');
    const ext = path.extname(file.originalname).slice(1).toLowerCase();
    if (allowedTypes.includes(ext)) {
      cb(null, true);
    } else {
      cb(new Error('File type is not allowed'));
    }
  }
});
```

Previous
**« Server Setup**

Next
**Controllers »**



Copyright © 2026 KUCCPS. Built with Docusaurus.